

Distributed [*Computing*] Systems

Load Balancing

Isaac D. Scherson (aka The Schark 😊)

Dept. of Computer Science (Systems)
Bren School of Information and Computer Sciences
University of California, Irvine
Irvine, CA 92697-3425

isaac@ics.uci.edu
www.ics.uci.edu/~isaac www.ics.uci.edu/~schark

CompSci-230, Spring 2019



© Isaac D. Scherson

1 / 45

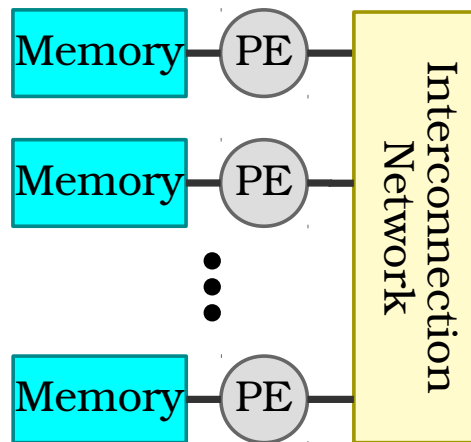
Load Balancing



© Isaac D. Scherson

2 / 45

Target Environment



Target Environment: MIMD. Multiple Tasks Per Processor.



Assumptions

- ▶ System assumed MIMD.
- ▶ Multiple jobs in system at any time.
- ▶ Jobs assumed parallel or sequential.
- ▶ Local Scheduling at each processor.
- ▶ Each Processor capable of self monitoring its performance.

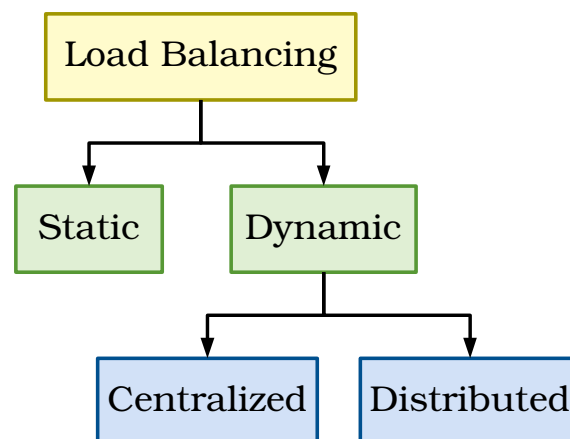


Load Balancing: Definition 1

- ▶ **Load Balancing** is the activity of migrating load units from one processing element (PE) to another so that the number of load units in different PEs is approximately the same.



Load Balancing: Classification



Load Balancing: Classification

- ▶ Static:
 - ▶ Task information (execution time) is computed from the application before load distribution. The allocation remains the same throughout the application's execution
- ▶ Dynamic:
 - ▶ No a priori task information is used in initial distribution. Must satisfy changing requirements by making task distribution decisions at run-time



Load Balancing: Classification

- ▶ Centralized:
 - ▶ Load Balancing decisions are made by one processing element only, which is responsible for maintaining global load information.
- ▶ Distributed:
 - ▶ Load Balancing decisions are made locally. Global load information is distributed among all processing elements and they all share the responsibility of achieving global load balance.



Load Balancing: Classification

- ▶ **Implicit:**
 - ▶ Load Balancing performed automatically by the system
- ▶ **Explicit:**
 - ▶ User decides which and when tasks should migrate



Load Balancing: Phases

- ▶ **When** to load balance
- ▶ **Where** to migrate the tasks
- ▶ **How many** tasks to transfer/request
- ▶ **Which** tasks to transfer



Load Balancing: Advantages

- ▶ Improves single application overall performance by redistributing the workload among processors.
- ▶ Increases throughput if multi-application environment.
- ▶ System function, hence transparent to user.



Load Balancing: Disadvantages

- ▶ Load Balancing activity comes at the expense of useful computation.
- ▶ Incurs into communications overhead:
 - ▶ Transfer of tasks among Pes.
 - ▶ Cost of collecting load information.
 - ▶ Message forwarding.
- ▶ Space required to maintain load balancing information.



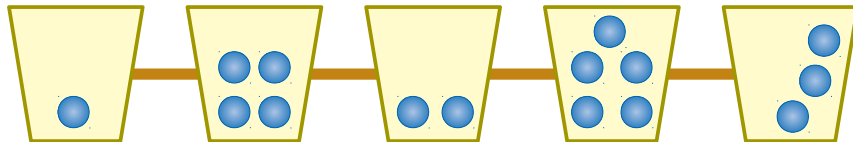
Load Balancing: Previous work

- ▶ Gradient Model
- ▶ Central Job Dispatcher
- ▶ Sender Initiated/Receiver Initiated Diffusion
- ▶ Randomize seeking
- ▶ Dimension Exchange Method
- ▶ Hierarchical Balancing Method



According to Definition 1

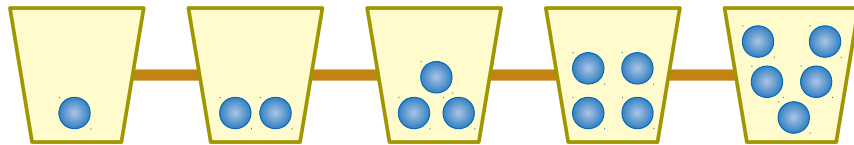
- ▶ Fully distributed algorithm:



- ▶ Can look at neighbors only and transfer marbles so that two neighbors have same number of marbles ± 1
- ▶ Converges to following configuration:



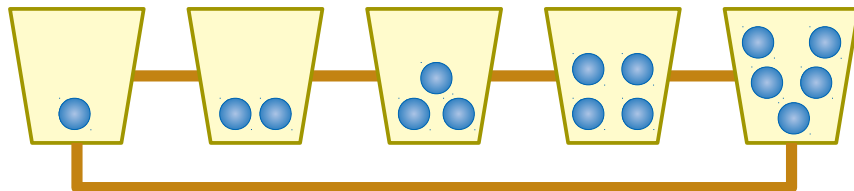
According to Definition 1



- ▶ Is this BALANCED ?
- ▶ Solution ?



According to Definition 1



Load Balancing: Definition 2

- ▶ **Load Balancing** is the activity of migrating load units from one processing element (PE) to another so that PEs have at least one load unit at all times (if enough workload is available).



- ▶ Redistribute workload among processors to increase throughput by maximizing processor utilization.
- ▶ Minimize processor idle time with minimum task redistribution overhead.



Load Balancing: Goals

- ▶ To minimize processor idling time.
- ▶ To minimize overhead:
 - ▶ Communications.
 - ▶ Load balancing activity.
- ▶ To be stable.
- ▶ To quickly (re-)distribute the load.



Rate-of-Change Load Balancing

- ▶ The load balancing algorithm proposed for:
 - ▶ High throughput
 - ▶ High Processor utilization
 - ▶ Fault tolerant
 - ▶ On-line repairable
 - ▶ Scalable



Rate of Change Load Balancing (RoC-LB)

- ▶ Dynamically compute the way in which local load changes in time (rate of change).
- ▶ Redistribute load if and only if future starvation is predicted.
- ▶ Assumes time slicing at each PE (time interval).



RoC-LB: Our Approach

- ▶ Uses local tables containing system's load distribution information to determine where tasks should be transferred to.
- ▶ Uses three thresholds to determine when and how many tasks to transfer.
- ▶ Uses task size and previous execution time to determine which tasks to transfer.



RoC-LB: Phases

- ▶ **When** to load balance
- ▶ **Where** to migrate the tasks
- ▶ **How many** tasks to transfer/request
- ▶ **Which** tasks to transfer



RoC-LB: When Phase

Each Processor

- ▶ Calculates its local rate of change (DL).
- ▶ Uses DL as predictor of the probable number of tasks in subsequent intervals.
- ▶ If DL is negative, computes number of sampling intervals before idle state reached.
- ▶ Initiates a migration request if number of intervals to starvation is less than network delay (ND).
- ▶ Two thresholds are used to determine the load status of a PE:
 - ▶ Above HT : Source PE
 - ▶ Below LT : Sink PE
 - ▶ Between HT and LT : Neutral PE
- ▶ Two exceptions:
 - ▶ When the load falls below CT
 - ▶ When the load is above HT

